

Q-Spire: Quantum Code Smell Detection and LLM-Assisted Remediation in VS Code

Riccardo Alfieri*, Francesco Paolo D’Antuono*, Stefano Lambiase†, Andrea De Lucia*, Fabio Palomba*

*Department of Computer Science—University of Salerno, Salerno, Italy

†Department of Computer Science—Aalborg University, Copenhagen, Denmark

Abstract—Quantum software engineering is maturing rapidly, and with it the concern that quantum-specific bad practices related to code—so-called *quantum code smells*—can quietly erode circuit reliability and maintainability. Existing detection tools work only from the command line, demand manual preprocessing of each script, and stop at reporting a count, leaving the developer with no concrete way forward. To fill the above-mentioned gap, we present Q-SPIRE, a Visual Studio Code extension that detects eight quantum-specific smells—defined in literature—through both static and dynamic analysis, surfaces them inline in the editor with user-configurable thresholds, and uses a Large Language Model (LLM) to produce contextual explanations and refactoring suggestions next to each detected smell. The tool runs on the developer’s active file with no manual preprocessing, supports modern Qiskit code, and exposes both an interactive GUI and a CLI for batch analysis. We illustrate Q-SPIRE on a set of open-source quantum repositories and report a preliminary think-aloud usability evaluation with master-level Software Engineering students.

Tool repository: <https://github.com/Riccardoalfieri2003/Q-Spire>

Tool video: <https://youtu.be/FLa-xl6wxko>

Index Terms—Quantum, Software Engineering, Code Smells, Technical Debt, IDE, Large Language Models.

I. INTRODUCTION

Open-source quantum programming frameworks, such as Qiskit, have made it possible for a wider population of developers to write executable quantum programs, and the number of public quantum repositories has been growing accordingly [1], [2]. As these projects grow in size and lifetime, they raise the same software engineering (SE) concerns that arise in classical software, including code understandability, maintainability, and quality assurance.

One specific concern in this space is the emergence of *quantum code smells*: recurring source-code patterns that, while not bugs, are associated with design or implementation problems specific to quantum software [3], [4]. Several of these patterns have no direct counterpart in classical software,

since they relate to constraints of the quantum execution model, such as decoherence, gate error, and qubit-topology limitations. Chen et al. [5] proposed an initial catalogue of eight quantum code smells—the “smelly eight”—together with a detection technique evaluated on a sample of Qiskit projects.

The tools currently available to detect these smells have practical limitations. QSMELL [5], the reference implementation, targets an older version of Qiskit, requires the user to write a wrapper script for each analysed circuit, and is exposed only through a command-line interface (CLI) that reports per-smell counts without information about smell locations or possible remediations. More recent learning-based proposals [6] address the detection problem at a methodological level and are not distributed as a reusable tool. As a consequence, quantum code smell analysis tends to be performed offline, by users with specific expertise, and outside the editor in which the quantum code is written.

In this paper, we present Q-SPIRE (*Quantum Smell Processing with Intelligent Remediation and Explanation*), a Visual Studio Code extension designed to enhance the practicality of quantum code smell analysis. Rather than introducing a new detection methodology, Q-SPIRE focuses on engineering an integrated workflow by providing three features that are not jointly supported by existing tools: (i) automatic detection of the eight quantum code smells in the catalog of Chen et al. [5], based on both static inspection and runtime simulation, with user-configurable thresholds and no manual preprocessing of the analyzed file; (ii) implementation as a plug-in of an integrated development environment (IDE), where files are analyzed on demand and detected smells are highlighted inline; and (iii) explanations and refactoring suggestions for each detected smell, generated through a Large Language Model (LLM) and shown alongside the detection report.

The paper makes three contributions. First, we describe the design and architecture of Q-SPIRE, including the dual static/dynamic detection back-end and the Factory-based explainer that decouples smell types from the corresponding LLM prompts. Second, we describe the integration of the tool into the development environment, covering active-file detection, inline highlighting, and threshold configuration through the settings panel. Third, we release Q-SPIRE as a publicly available tool—that we are already empirically evaluating—to support reuse and replication.

This work has been partially supported by the project QUASAR (grant 2022T2E39C), under the PRIN 2022 MUR program (EU NextGenerationEU).

Accepted version. This paper has been accepted for publication in the Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME) 2026; the final version of record will appear in IEEE *Xplore* (DOI to be added upon publication). © 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

II. BACKGROUND AND RELATED WORK

Quantum software engineering (QSE) extends classical SE practices to programs that target quantum hardware [7]–[12]. Qiskit is among the most widely adopted frameworks in the open-source ecosystem, and recent empirical studies have started to characterise the practices, communities, and challenges of quantum software developers [1], [13].

Technical debt (TD) refers to the long-term cost of sub-optimal design or implementation decisions made during development [14], [15]. TD has been associated with reduced maintainability, higher change cost, and effects on developer productivity [16]–[18]. Code smells [18] are one of the most studied symptoms of TD, supported by an extensive literature on detection and impact in classical software [19]–[21].

TD also manifests in quantum software, where it interacts with the constraints of current quantum hardware—e.g., decoherence, gate error, and limited qubit topology—in ways that classical heuristics do not directly capture. Recent studies report the presence of TD-related issues in open-source quantum projects [3], motivating the development of quantum-specific analysis techniques.

The first catalogue of quantum-specific code smells—the “smelly eight”—was proposed by Chen et al. [5]. The catalogue is grounded in Qiskit programming practices and is accompanied by QSMELL, a detection tool that the authors validated on a small sample of open-source projects. QSMELL has practical limitations for daily use: it depends on an outdated Qiskit version, requires the user to provide a wrapper script for each analysed circuit, uses fixed detection thresholds, and exposes only a CLI that reports smell counts without indicating their location or suggesting remediations.

Malhotra et al. [6] addressed quantum code smell detection from a learning-based perspective, training neural models on labelled smell data. Their contribution focuses on the methodological evaluation of the proposed approach, and the resulting models are not packaged as a reusable tool.

In summary, the tools currently available for quantum code smell detection have been developed primarily as research artifacts, supporting the validation of a definition or a model rather than routine use by quantum developers. None of them combines automated detection of the eight smells, IDE integration, and remediation support. Q-SPIRE addresses this gap: it packages the detection logic into an IDE-integrated tool, removes the manual preprocessing step required by prior work, and provides LLM-based remediation alongside detection. Targeting Qiskit as the prominent framework in the field, Q-Spire is tailored for quantum developers and researchers within this rapidly growing, emerging ecosystem.

III. THE Q-SPIRE TOOL

A. Goals and Requirements

The design of Q-SPIRE is driven by four requirements: **R1—Coverage**: the tool supports all eight smells defined by Chen et al. [5] without redefining them, so that its detection results remain comparable to existing studies; **R2—Automation**: any Qiskit file can be analyzed as-is, removing

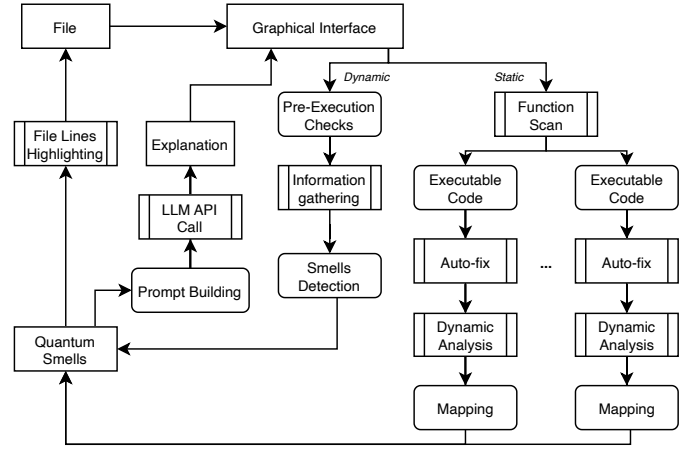


Fig. 1. Architecture of the tool Q-Spire.

the manual wrapper step required by QSMELL; **R3—IDE integration**: detection results are surfaced inline in the developer’s editor, to limit context switching; **R4—Actionability**: detection is paired with concrete remediation suggestions, rather than reporting smell counts in isolation.

B. Architecture Overview

Q-SPIRE is structured around three components (Figure 1): a *Detector* layer, an *Explainer* layer, and a *Presentation* layer. Both the Detector and the Explainer layers follow the Factory design pattern so that introducing a new smell type is easy.

1) *Smell Taxonomy*: All smells inherit from an abstract `QuantumSmell` class that carries the common metadata required to localize and report a detection: file path, line, column, smell type, and the threshold context used at detection time. Each of the eight smells is then implemented as a concrete subclass that adds smell-specific attributes, such as the qubit index for `IdQ` or the matrix shape for `CG`.

2) *Detection Back-End*: Detection runs in two complementary modes, selected at invocation time through the `-method` flag of the `qspire` CLI. Independently of the mode, two of the eight smells—`LPQ` and `CG`—are detected by inspecting the source code for structural patterns: calls to `transpile()` without `initial_layout` (`LPQ`) and uses of `UnitaryGate`, `HamiltonianGate`, or `SingleQubitUnitary` (`CG`). The remaining six smells (`ROC`, `NC`, `LC`, `IM`, `IdQ`, `IQ`) require access to a representation of the quantum circuit defined in the analysed file; the two modes differ only in how this representation is obtained.

In the **static** mode, Q-SPIRE reconstructs each circuit by *simulation*, without running the user’s program as-is. The tool generates a synthetic main block, mocks unknown dependencies, and executes the resulting script under controlled conditions. An auto-fixing loop handles common runtime errors, such as missing imports or uninitialised variables, up to a bounded number of iterations, and a two-stage line-mapping process maps simulation findings back to the corresponding lines in the user’s source file. This mode is suited to files that

cannot be executed in isolation, for example modules that lack an entry point or that depend on a larger project context.

In the **dynamic** mode, Q-SPIRE executes the user’s program directly within a sandbox that performs pre-execution safety checks (e.g., refusal to run code that contains `exec()`) and runtime monitoring (e.g., call-depth bounds to prevent infinite recursion). As the program runs, all quantum operations are recorded, producing a trace of gate types, target qubits, and source locations that the smell detectors consume. This mode is suited to files that are self-contained and ready to execute.¹

In both modes, the resulting circuit representation is consumed by the same detector hierarchy, so that detection rules are written once and produce comparable outputs regardless of how the representation was obtained.

3) *LLM-Assisted Explainer*: The Explainer hierarchy mirrors the Detector hierarchy: each smell type has a dedicated explainer that builds a smell-specific prompt, combining a description of the smell, the detection metric, and the concrete instance (location, code snippet, and computed values).² The base Explainer routes the prompt to an LLM through the OpenRouter application programming interface (API). The default model is a free model, chosen for its zero-cost availability and acceptable latency; users can switch to any OpenRouter-compatible model from the settings panel of the extension.

4) *Presentation Layer*: Two front-ends share the same back-end layer. The **CLI** (`qspire -method <static|dynamic> <path>`) supports batch analysis of single files or whole repositories, producing one CSV per analyzed file; this mode is intended for researchers and developers running large-scale or scripted analyzes. The **VS Code extension** provides the interactive experience: a single click on the toolbar icon analyzes the currently active file, highlights smelly regions inline, and opens a side panel that lists the detected smells with their location and computed metric, supports per-smell threshold editing, and triggers on-demand LLM explanations, offering recommendations developers should review for potential incorrect suggestions.

C. Detection Rules

Table I summarises the detection rule and the default threshold used by Q-SPIRE for each of the eight smells. The rules implement the definitions of Chen et al. [5] where directly applicable, with minor adjustments to accommodate API changes in recent Qiskit versions (e.g., the deprecation of `bind_parameters` in favour of `assign_parameters` for NC). The table is provided to make the paper self-contained; further details are available in the tool repository.

All thresholds in Table I are user-editable: the GUI exposes them in the settings panel of the VS Code extension, and the CLI reads the same values from a `config.json` file shipped

¹Together, the simulation step of the static mode and the direct execution of the dynamic mode remove the manual wrapper step required by QSMELL [5], where the user has to provide a custom entry point for each analysed circuit.

²The same prompt also asks the LLM for a candidate refactoring, grounded in the smell-specific context. These suggestions are not validated by Q-SPIRE and should be treated as insights that the developer is expected to review and, if appropriate, implement.

with the tool. This addresses a known limitation of QSMELL, where thresholds are hard-coded in the source and cannot be adjusted without modifying the tool itself.

IV. USE AND INSTALLATION

This section describes how a developer interacts with Q-SPIRE (which GUI is shown in Figure 2) in practice.

A. Installation

Q-SPIRE requires Python 3.8–3.13 and VS Code version 1.60 or later. Installation involves two steps. First, the user clones the tool repository and runs `pip install -e .` to install the Python back-end and register the `qspire` CLI on the system. Second, the bundled `qspire-0.0.1.vsix` package is installed in VS Code through the command palette, which adds the extension and its toolbar icon to the editor. On first launch, the extension prompts the user for an OpenRouter API key, which is stored in the settings panel; this key is required only by the LLM-based explanation feature, while detection and threshold customization work without it.

B. Usage Scenario: Detecting and Refactoring an IdQ Smell

We illustrate the typical workflow on a developer who is editing `circuit_demo.py`, a Qiskit script that builds a small variational ansatz. The developer presses the Q-SPIRE icon on the VS Code activity bar and selects an analysis mode (static or dynamic). The extension picks up the currently active file automatically, without requiring a file selection dialog, and runs the selected detector on it. Once the analysis completes, the editor highlights the lines that triggered a detection through inline diagnostic underlines.

Hovering on a highlighted line displays a brief warning indicating the smell type and its location—for example, an IdQ warning reporting that qubit `q[2]` is left idle between two gates. The side panel of the extension lists all detected smells together with their file, line, smell type, and the metric value computed by the detector. Selecting a smell and clicking the “Explain” button triggers the Explainer, which sends the smell-specific prompt to OpenRouter and returns a textual explanation of the detected pattern, together with a refactoring suggestion such as reordering the gate sequence or inserting an identity bridge.

The settings panel of the extension exposes the detection thresholds as numeric inputs, one per smell. The developer can adjust a threshold—for example, to require a longer idle interval before triggering an IdQ warning—and re-run the analysis from the same panel; the editor updates the set of detections without requiring a restart.

C. Batch Analysis via CLI

For repository-level analysis or integration into continuous-integration pipelines, the same back-end is exposed through the CLI. The command `qspire -method <static|dynamic> <path> <out_dir>` analyses the file or directory passed as `<path>` and writes one CSV file per analysed source file into `<out_dir>`, with one row per

TABLE I
DETECTION RULES AND DEFAULT THRESHOLDS FOR THE EIGHT QUANTUM-SPECIFIC CODE SMELLS.

Smell	Definition	Detection technique in Q-Spire	Default threshold
CG	Use of customized gates	Static scan for <code>UnitaryGate</code> , <code>HamiltonianGate</code> , <code>SingleQubitUnitary</code> .	Presence-based
ROC	Repeated set of operations	Circuit-level analysis identifying recurring sub-sequences across the whole circuit.	Sub-circuit length ≥ 1
NC	Non-parameterized circuit	Static comparison between calls to <code>run/execute</code> and <code>assign_parameters/bind_parameters</code> .	Execution dominates binding
LC	Long circuit	$(1 - error)^{l \cdot c}$ with l = max ops on any qubit, c = max parallelism.	$error = 0.03512$, threshold < 0.5
IM	Intermediate measurements	Detection of measurement operations applied before circuit termination.	Presence-based
IdQ	Idle qubits	Distance, in operations, between consecutive gates on each qubit.	Idle distance ≥ 2
IQ	Init. of qubits $\neq 0\rangle$	Distance between first and second operation on each qubit after initialisation.	Distance ≥ 2
LPQ	Logical/physical misalignment	Static scan for <code>transpile()</code> calls without <code>initial_layout</code> .	Presence-based

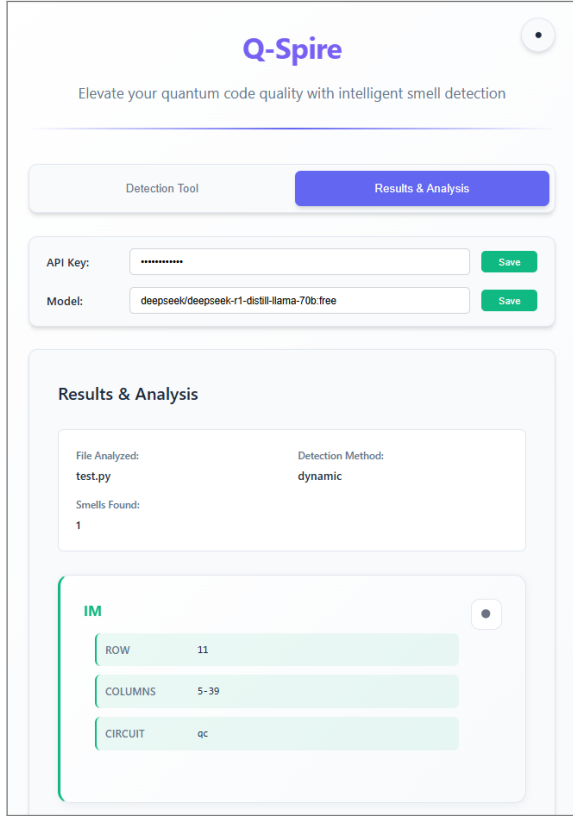


Fig. 2. Results Screen of the tool QSpire.

detected smell. Detection thresholds, the OpenRouter API key, and the chosen LLM are read from the same `config.json` file used by the extension, so the same configuration can be reused across the two front-ends.

V. EMPIRICAL EVIDENCE

We complement the description of Q-SPIRE with two pieces of empirical evidence. Consistent with the scope of a tool demonstration paper, the goal is not a full-scale evaluation, but to show that Q-SPIRE produces meaningful results on real code and is usable by its target audience.

A. Descriptive Smell Prevalence

We applied Q-SPIRE to a curated dataset of 43 open-source quantum repositories. The repositories were selected through standard criteria for mining studies on the quantum software ecosystem: each project is publicly available on GitHub, written in Python, and based on Qiskit; has at least 50 commits over at least six months of activity; and has a minimum level of community engagement, used as a proxy for non-trivial quantum projects. The detector was run in static mode on the latest snapshot of each repository, and detections were aggregated at the repository level.

All eight smells are observed in at least a subset of the analysed repositories, with a markedly uneven distribution. *LPQ* is by far the most prevalent smell, occurring in more than three out of four repositories (76.2%); *LC* and *ROC* follow with 45.2% and 42.9%, respectively. A second group of smells occurs in roughly one third of the repositories: *CG* (35.7%), *NC* (33.3%), and *IdQ* (33.3%). Lower prevalence values are observed for *IM* (26.2%) and, in particular, *IQ* (16.7%), the least prevalent smell in the dataset. As an additional sanity check, the detection back-end has been compared against the original implementation by Chen et al. [5] on a shared subset of repositories, with nearly identical detection results. The prevalence values we observe are also of the same order of magnitude as those reported by Chen et al. [5].

B. Think-Aloud Evaluation

We complemented the prevalence analysis with a preliminary usability evaluation conducted as a think-aloud test [22], a qualitative protocol in which participants verbalize their thoughts while interacting with the system. The method is suited to early-stage tools, since it surfaces interaction issues with a small number of participants.

We defined two tasks: (T1) installing Q-SPIRE from scratch on the participant's machine, and (T2) analyzing a set of provided Qiskit scripts and reading the resulting report, with the goal of assessing whether the tool supports a general understanding of the detected smells. We recruited five participants at the institution of the third author: all are master-level students in Software Engineering who had previously attended an introductory course on quantum computing.

In T1, three out of five participants completed the installation without issues. The remaining two encountered conflicts between the tool’s required libraries and the Python environment already installed on their machine; in both cases, resetting the environment and reinstalling solved the problem.

In T2, all participants reported that the LLM-generated explanations were clearer and more useful than reading the smell definitions in a static table or report, which was provided as supporting documentation. Participants also rated the GUI as clear and easy to use. As suggestions for future evolution, they pointed to two extensions: a project-level dashboard that aggregates detections across an entire repository, rather than per-file, and the addition of a GUI front-end for the mining-oriented CLI.

VI. DISCUSSION AND IMPACT

Research impact. By automating the analysis of arbitrary Qiskit files and exposing the detection thresholds as user-configurable parameters, Q-SPIRE removes two recurring obstacles to large-scale empirical studies on quantum-specific smells: the need for manual preprocessing of each analysed file, and the rigidity of fixed detection rules. These features support reproducible mining and longitudinal analyses across large samples of quantum repositories, and allow researchers to adapt the analysis to context-specific definitions of what counts as a smell.

Practitioner impact. For developers, the integration of detection and remediation into the editor makes smell analysis available during development rather than as a separate auditing activity. The LLM-generated explanations and refactoring suggestions reduce the cost of acting on a detection, especially for developers who are familiar with Qiskit but less familiar with the design rationale behind a specific smell.

Educational impact. The LLM-based explainer can also be used in educational and training settings, where newcomers to quantum programming may benefit from a contextual description of why a given pattern is considered a smell and from concrete suggestions to address it.

Limitations. The current implementation has three known limitations. First, the dynamic detection mode requires the analysed program to be executable, which limits its applicability to scripts that have all their runtime dependencies satisfied. Second, the LLM-based explainer inherits the well-known failure modes of large language models, including factual hallucinations and limited knowledge of the most recent Qiskit releases; the validation of the generated explanations is itself an open research problem. Third, the auto-fix loop used by the static mode covers a bounded set of common runtime errors, so files that exercise less frequent error patterns may still fail to produce a usable simulation.

VII. CONCLUSION AND FUTURE WORK

We presented Q-SPIRE, a VS Code extension for quantum code smell analysis. The tool detects the eight smells defined by Chen et al. [5] on modern Qiskit code without requiring manual preprocessing, exposes both a GUI inside the editor

and a CLI for batch analysis, and pairs each detection with an LLM-generated explanation and refactoring suggestion.

We plan to extend Q-SPIRE along three directions. First, we will broaden the coverage of the auto-fix loop used by the static mode, so that a wider range of runtime errors can be handled without aborting the analysis. Second, we will work on the validation of the LLM-generated explanations, both through expert assessment and through prompt engineering aimed at reducing factual errors. Third, we will act on the suggestions that emerged from the think-aloud evaluation, in particular by adding a project-level dashboard for repository-wide detections and a graphical front-end for the mining-oriented CLI, and by refining the default detection thresholds with feedback from a larger and more diverse pool of users.

REFERENCES

- [1] M. De Stefano *et al.*, “The quantum frontier of software engineering: A systematic mapping study,” *Information and Software Technology*, p. 107525, 2024.
- [2] M. Piattini *et al.*, “Quantum computing: A new software engineering golden age,” *ACM SIGSOFT Software Engineering Notes*, vol. 45, no. 3, pp. 12–14, 2021.
- [3] M. Openja *et al.*, “Technical debts and faults in open-source quantum software systems: An empirical study,” *Journal of Systems and Software*, vol. 193, p. 111458, 2022.
- [4] M. De Stefano *et al.*, “An empirical study into the effects of transpilation on quantum circuit smells,” *Empirical Software Engineering*, vol. 29, no. 3, p. 61, 2024.
- [5] Q. Chen *et al.*, “The smelly eight: An empirical study on the prevalence of code smells in quantum computing,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 358–370.
- [6] R. Malhotra *et al.*, “Qnet: exploring deep learning for quantum code smell detection,” *Software Quality Journal*, vol. 33, no. 2, pp. 1–28, 2025.
- [7] M. Piattini *et al.*, “Toward a quantum software engineering,” *IT Professional*, vol. 23, no. 1, pp. 62–66, 2021.
- [8] M. De Stefano *et al.*, “Software engineering for quantum programming: How far are we?” *Journal of Systems and Software*, vol. 190, p. 111326, Aug. 2022.
- [9] A. Ahmad *et al.*, “Towards process centered architecting for quantum software systems,” in *2022 IEEE international conference on quantum software (QSW)*. IEEE, 2022, pp. 26–31.
- [10] S. Lambiase and A. De Lucia, “Quantifying adoption: A sem study of quantum software technology in software development,” *Empirical Software Engineering*, 2026.
- [11] S. Lambiase *et al.*, “Socio-technical well-being of quantum software communities: An overview on community smells,” in *Euromicro Conference on Software Engineering and Advanced Applications*. Springer, 2025, pp. 39–56.
- [12] A. Della Porta *et al.*, “Are large language models ready for quantum software engineering? a multivocal literature review,” 2026.
- [13] M. De Stefano *et al.*, “Quantum software engineering issues and challenges: Insights from practitioners,” in *Quantum Software: Aspects of Theory and System Design*. Springer, 2024, pp. 337–355.
- [14] W. Cunningham, “The wycash portfolio management system,” *ACM Sigplan Oops Messenger*, vol. 4, no. 2, pp. 29–30, 1992.
- [15] P. Avgeriou *et al.*, “Managing technical debt in software engineering (dagstuhl seminar 16162),” *Dagstuhl reports*, vol. 6, no. 4, pp. 110–138, 2016.
- [16] Z. Li *et al.*, “A systematic mapping study on technical debt and its management,” *J. Syst. Softw.*, vol. 101, pp. 193–220, 2015.
- [17] W. Behutiye *et al.*, “Analyzing the concept of technical debt in the context of agile software development: A systematic literature review,” *Inf. Softw. Technol.*, vol. 82, pp. 139–158, 2017.
- [18] F. Palomba *et al.*, “Beyond Technical Aspects: How Do Community Smells Influence the Intensity of Code Smells?” *IEEE Transactions on Software Engineering*, vol. 47, no. 1, pp. 108–129, Jan. 2021.

- [19] N. Moha *et al.*, “Decor: A method for the specification and detection of code and design smells,” *IEEE Transactions on Software Engineering*, vol. 36, no. 1, pp. 20–36, 2009.
- [20] F. Palomba *et al.*, “Mining version histories for detecting code smells,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 462–489, 2014.
- [21] F. Pecorelli *et al.*, “A large empirical assessment of the role of data balancing in machine-learning-based code smell detection,” *Journal of Systems and Software*, vol. 169, p. 110693, 2020.
- [22] J. Nielsen, “Estimating the number of subjects needed for a thinking aloud test,” *International journal of human-computer studies*, vol. 41, no. 3, pp. 385–397, 1994.